

Estado del Arte

Modelado de Joyería en Tiempo Real mediante Realidad Aumentada Multiplataforma

Valentina Carreño - Juan Esteban Garzón - José Ontiveros - André Landinez

Planeación proyecto de grado Grupo 7 · Marzo 2026

1. Introducción y Contexto

El comercio digital de joyería de lujo enfrenta una paradoja estructural: mientras que el 73% de las compras comienzan en dispositivos móviles, la decisión final de compra depende de factores sensoriales y físicos como el ajuste de la pieza, el brillo de los materiales y las proporciones reales sobre el cuerpo. Esta brecha entre la experiencia digital y la experiencia en tienda genera altas tasas de abandono de carrito y devoluciones.

La Realidad Aumentada (AR) combinada con técnicas de procesamiento de imágenes en tiempo real surge como la solución más prometedora para cerrar esta brecha. El mercado global de aplicaciones de prueba virtual (Virtual Try-On) de joyería, valorado en 1.24 mil millones de dólares en 2024, proyecta una expansión hacia los 4.11 mil millones para 2033, impulsado por una tasa de crecimiento anual compuesta (CAGR) del 17.2%.

Este documento presenta el estado del arte de las tecnologías relevantes para construir una aplicación multiplataforma que permita a los usuarios visualizar anillos y aretes sobre su propio cuerpo en tiempo real usando la cámara de su celular.

Tabla 1. Impacto económico y de rendimiento de la AR en joyería

Indicador	Valor / Impacto
Tamaño del mercado AR Joyería (2033)	\$4.11 mil millones USD
CAGR del sector	17.2% anual
Incremento en tasa de conversión (Shopify)	Hasta +94%
Reducción de devoluciones de productos	Hasta -40%
Aumento en satisfacción del cliente	+32%
Compras iniciadas en móvil	73% del total

2. Enfoques de Desarrollo: Nativo vs. Multiplataforma

2.1 Desarrollo Nativo (Android / iOS)

La recomendación más frecuente en foros y tutoriales especializados es desarrollar aplicaciones de AR directamente en los frameworks nativos: ARCore para Android (usando Kotlin o Java) y ARKit para iOS (usando Swift). Este enfoque ofrece acceso completo a las APIs de cada plataforma, mejor rendimiento en hardware específico y soporte oficial sin intermediarios.

Sin embargo, el desarrollo nativo implica mantener dos bases de código independientes, duplicando el esfuerzo de desarrollo, las pruebas y el mantenimiento. Para un proyecto académico o un MVP (producto mínimo viable), esto representa una barrera significativa de tiempo y recursos.

2.2 Flutter como Alternativa Multiplataforma

Flutter, el framework de Google para el desarrollo multiplataforma, ofrece una alternativa viable para proyectos que buscan paridad funcional en iOS y Android desde una sola base de código. La pregunta central de esta investigación es: ¿puede Flutter soportar una experiencia de AR con procesamiento de imágenes en tiempo real para joyería?

La respuesta corta es: sí, aunque con ciertas consideraciones. Flutter no incluye capacidades de AR nativas, pero se comunica con ARKit y ARCore a través de plugins especializados. Esto significa que el motor de AR subyacente sigue siendo nativo en cada plataforma, y Flutter actúa como capa de presentación y lógica de negocio.

Ventajas de Flutter para este proyecto

- Una sola base de código para iOS y Android, reduciendo el tiempo de desarrollo a la mitad.
- Hot Reload: permite ver cambios en tiempo real sin reiniciar la aplicación, muy útil para ajustar la visualización de joyas.
- Ecosistema de widgets rico para construir la interfaz de usuario (catálogo de joyas, selector de modelos, etc.).
- El motor de renderizado Impeller (disponible desde Flutter 3.x) mejora la fluidez del renderizado 3D al precompilar shaders, reduciendo los saltos de cuadros (jank) que arruinarían la experiencia AR.

Limitaciones a considerar

- Flutter no es la primera opción para AR de alta fidelidad. Frameworks como Unity o aplicaciones 100% nativas ofrecen mayor control y rendimiento.
- Los plugins de AR para Flutter dependen de mantenimiento de la comunidad, no del equipo oficial de Google o Apple.
- Para efectos visuales muy complejos (ray-tracing en tiempo real, simulación física de joyas), Flutter puede quedarse corto comparado con motores dedicados.

3. Plugins de AR Disponibles en Flutter

El ecosistema de Flutter cuenta con varias opciones para integrar AR. A continuación se presentan las más relevantes para este proyecto:

Tabla 2. Comparativa de plugins AR para Flutter

Plugin	Plataformas	Observación principal
ar_flutter_plugin	iOS + Android	El más completo y usado. Soporta ARKit y ARCore, modelos GLB/GLTF, anclas y detección de planos.
ar_flutter_plugin_2	iOS + Android	Fork actualizado que reemplaza Sceneform (archivado por Google) con SceneView y Filament como motor 3D.
arkit_flutter_plugin	iOS solamente	Acceso directo a ARKit. Más funciones en Apple pero rompe la multiplataforma.
arcore_flutter_plugin	Android solamente	Acceso directo a ARCore. Similar al anterior pero para Android.
flutter_cube	iOS + Android	Renderizado 3D básico con archivos OBJ. No es AR propiamente, pero sirve para visualización de productos.

Para este proyecto, `ar_flutter_plugin` o su fork `ar_flutter_plugin_2` son las opciones más recomendadas, ya que mantienen la premisa multiplataforma y permiten cargar modelos 3D en formato GLB, que es el formato estándar de los gemelos digitales de joyería.

4. Procesamiento de Imágenes: Detección Corporal en Tiempo Real

El núcleo técnico del proyecto es la capacidad de detectar con precisión la posición de los dedos (para anillos) y las orejas (para aretes) en cada fotograma del video en tiempo real. Para esto existen dos grandes familias de soluciones:

4.1 MediaPipe de Google

MediaPipe es la solución más madura y ampliamente usada para este tipo de proyectos. Es una biblioteca de machine learning de código abierto desarrollada por Google, optimizada para correr en dispositivos móviles con baja latencia. Sus módulos más relevantes para joyería son:

- **MediaPipe Hands:** Detecta 21 puntos de referencia (landmarks) 3D por cada mano en tiempo real. Usa una CNN optimizada para móviles que primero detecta la palma y luego estima las posiciones de cada articulación de los dedos. Para anillos, los puntos de los nudillos permiten anclar el modelo 3D con precisión.
- **MediaPipe Face Mesh:** Detecta 468 puntos en el rostro, incluyendo puntos cercanos al lóbulo de la oreja. Permite anclar aretes virtuales que se muevan con la cabeza del usuario.
- **Disponibilidad en Flutter:** Existen integraciones como el plugin de ThinkSys y el repositorio JaeHeee/FlutterWithMediaPipe que demuestran la viabilidad de usar MediaPipe dentro de una app Flutter.

4.2 ML Kit de Google (Firebase)

ML Kit ofrece modelos de detección de rostros y pose corporal que se integran fácilmente en Flutter mediante el paquete `google_mlkit_face_detection`. Aunque es menos preciso que MediaPipe para seguimiento de manos, su integración con el ecosistema Firebase lo hace atractivo para proyectos que ya usan servicios de Google.

4.3 ARKit y ARCore Nativos

Tanto ARKit (iOS) como ARCore (Android) incluyen capacidades propias de seguimiento de manos y rostros. ARKit, en particular, tiene un modo de detección de manos muy preciso en dispositivos Apple recientes. Sin embargo, estas APIs son de acceso limitado desde Flutter, siendo más fácil acceder a ellas desde el código nativo de cada plataforma.

Tabla 3. Comparativa de capacidades de detección corporal

Tecnología	Detección de manos	Detección facial / orejas
MediaPipe Hands	21 puntos 3D por mano, alta precisión	No incluido (usar Face Mesh)
MediaPipe Face Mesh	No incluido (usar Hands)	468 puntos, incluye zona de orejas
ML Kit Pose	Puntos básicos de muñeca	Puntos faciales básicos
ARKit (nativo iOS)	Seguimiento de manos de alta precisión	Face tracking avanzado
ARCore (nativo Android)	Seguimiento limitado de manos	Face tracking mediante ARCore ML

5. Modelos 3D de Joyas y Renderizado Realista

5.1 Formatos de Modelos 3D

Para que una joya virtual luzca convincente, el modelo 3D debe cumplir dos condiciones: geometría precisa y materiales realistas. El formato estándar para aplicaciones de tiempo real

es GLB/gLTF (GL Transmission Format), desarrollado por el consorcio Khronos. Este formato es ligero, eficiente y soporta materiales PBR (Physically Based Rendering), que es la técnica que permite simular cómo la luz interactúa con metales y gemas.

Los modelos CAD de joyería (creados en herramientas como Rhinoceros, JewelCAD o Matrix) se exportan a GLB para su uso en AR. El proceso incluye: modelado de geometría, optimización de mallas para GPU móviles, y configuración de materiales PBR con parámetros de metalicidad, rugosidad y oclusión ambiental.

5.2 Renderizado PBR (Physically Based Rendering)

PBR es la técnica de renderizado que define cómo se comporta la luz sobre un material. Para joyería esto es crítico: un anillo de oro de 18k, uno de plata y uno de platino se ven completamente diferentes no solo por su color, sino por cómo reflejan y dispersan la luz. Los parámetros PBR clave son:

- Metalicidad (Metallic): valor de 0 a 1 que indica si el material es metálico o dieléctrico.
- Rugosidad (Roughness): define qué tan pulida es la superficie. Un anillo brillante tiene rugosidad cercana a 0.
- Color base (Albedo/Base Color): el color del material sin iluminación.
- Oclusión ambiental (Ambient Occlusion): simula sombras en grietas y uniones entre piezas.

Para las piedras preciosas (diamantes, rubíes, esmeraldas), se requieren shaders adicionales que simulen la refracción y dispersión de la luz. Cada tipo de gema tiene un Índice de Refracción (IOR) diferente: el diamante tiene IOR de 2.417, que genera su característico brillo y destellos arcoíris.

5.3 Herramientas de Renderizado 3D en Flutter

Para integrar modelos GLB en una app Flutter con renderizado PBR, las opciones actuales son:

- flutter_3d_controller: Paquete para Flutter que permite cargar y mostrar modelos GLB/GLTF con animaciones. Usa el widget ModelViewer internamente.
- model_viewer_plus: Wrapper de Flutter para el componente web <model-viewer> de Google, que soporta PBR y AR básica.
- Three.js (vía WebView): Para experiencias web embebidas, Three.js es la biblioteca estándar para gráficos 3D en JavaScript. Puede integrarse en Flutter a través de un WebView, aunque con penalización de rendimiento.
- Motor Impeller de Flutter: El motor de renderizado nativo de Flutter, disponible desde la versión 3.x, mejora la fluidez del renderizado 3D al precompilar los shaders, evitando los saltos de cuadros que arruinarían la experiencia de prueba virtual.

6. Retos Técnicos del Proyecto

6.1 Oclusión Realista

Uno de los problemas más difíciles es la oclusión: hacer que la parte trasera de un anillo quede tapada por el dedo del usuario. Sin sensores de profundidad (como el LiDAR de iPhones Pro), el sistema debe inferir la profundidad con técnicas heurísticas como:

- Modelado cilíndrico del dedo: se asume que el dedo es un cilindro y se recortan las partes del anillo virtual que quedarían detrás de ese cilindro.
- Segmentación por tono de piel: se detectan los píxeles de piel y se usa esa máscara para ocultar partes del modelo virtual.
- Redes neuronales para predicción de oclusión: enfoques más avanzados usan modelos de ML que predicen la posición de articulaciones ocultas.

6.2 Estabilidad del Tracking (Jitter)

El ruido natural de la cámara y los pequeños movimientos involuntarios del usuario hacen que los puntos de referencia detectados tiemblen entre fotogramas, lo que se manifiesta como un anillo o arete que vibra visualmente. Las soluciones incluyen filtros de suavizado como el filtro de Kalman o el suavizado exponencial, que promedian las posiciones detectadas en los últimos fotogramas.

6.3 Escalado y Proporciones

Para que un anillo virtual parezca real, debe tener el tamaño correcto relativo al dedo del usuario. Sin una referencia de escala física, el sistema debe estimarla a partir del tamaño percibido de la mano en el fotograma. Técnicas como la estimación de distancia focal y la antropometría estadística (tamaño promedio de dedos por tipo de mano) se usan para aproximar el tamaño correcto.

6.4 Rendimiento en Dispositivos de Gama Media

Correr detección de puntos de referencia por ML y renderizado 3D con PBR simultáneamente puede sobrecargar dispositivos de gama media. Las optimizaciones incluyen: reducir la resolución del frame analizado por MediaPipe, limitar la frecuencia de actualización del tracking, usar modelos 3D optimizados con bajo número de polígonos, e implementar instanciado de mallas para joyas con múltiples gemas idénticas.

7. Soluciones Comerciales de Referencia

Antes de construir desde cero, vale la pena conocer las soluciones que ya existen en el mercado. Algunas de las más relevantes son:

- mirrAR: Plataforma especializada en AR para joyería. Ofrece prueba virtual de anillos, aretes y pulseras con alta precisión. Usada por marcas como Tanishq y Giva.
- Banuba Face AR SDK: SDK comercial de AR facial con soporte para Flutter. Permite añadir joyas virtuales al rostro con seguimiento facial preciso.
- Snap Lens Studio: Herramienta de Snapchat que incluye un módulo específico para renderizado de gemas con shaders de refracción configurables. Aunque está atada al ecosistema de Snap, sus implementaciones técnicas son referencia para el estado del arte.
- Shopify AR: Integración de AR en tiendas Shopify que usa QuickLook (iOS) y Scene Viewer (Android) para mostrar productos 3D. Reporta incrementos de hasta 94% en conversión.

Estas soluciones demuestran que el problema es técnicamente solucionable, pero la mayoría son de código cerrado y costo elevado, lo que justifica el desarrollo de una solución propia de código abierto con Flutter.

8. Stack Tecnológico Propuesto

Basado en el estado del arte revisado, la arquitectura tecnológica recomendada para este proyecto es la siguiente:

Tabla 4. Stack tecnológico recomendado

Capa	Tecnología	Función
Framework principal	Flutter (Dart)	Base de código multiplataforma iOS/Android
Motor AR	ar_flutter_plugin_2	Integra ARKit (iOS) y ARCore (Android)
Detección de manos	MediaPipe Hands	21 puntos 3D para anillos
Detección facial/orejas	MediaPipe Face Mesh	468 puntos para aretes
Modelos 3D	GLB / glTF + PBR	Gemelos digitales de las joyas
Renderizado 3D	Flutter Impeller + Three.js	Shaders metálicos y de gemas
Filtrado de movimiento	Filtro de Kalman	Suavizado del tracking
Herramientas CAD	Rhinoceros / JewelCAD	Creación de modelos de joyas

9. Conclusiones

El estado del arte muestra que desarrollar una aplicación de AR para prueba virtual de joyería en Flutter es técnicamente viable. Flutter cuenta con los plugins necesarios para acceder a ARKit y ARCore desde una sola base de código, y MediaPipe ofrece una solución robusta y gratuita para la detección de manos y rostro en tiempo real.

Los principales retos del proyecto no son la disponibilidad de las tecnologías, sino su integración adecuada: lograr que el tracking sea estable, que la joya virtual tenga el tamaño correcto, y que el renderizado sea lo suficientemente realista para convencer al usuario. Estos son problemas que la industria ha resuelto en plataformas comerciales y que en el contexto académico pueden abordarse con soluciones aproximadas pero funcionales.

La decisión de usar Flutter sobre un enfoque nativo puro es un intercambio razonable para este proyecto: se pierde algo de rendimiento y profundidad de acceso a las APIs nativas, pero se gana la capacidad de llegar a ambas plataformas con un equipo de desarrollo más pequeño y en menos tiempo.

Como trabajo futuro, la solución puede evolucionar hacia el uso de inteligencia artificial generativa para personalización de diseños en tiempo real, renderizado en la nube para mayor calidad visual, y anclas persistentes que permitan guardar y compartir pruebas virtuales entre usuarios.

10. Alianzas Estratégicas con Joyerías Locales

Uno de los factores diferenciadores de este proyecto frente a otras iniciativas académicas similares es el acceso a piezas físicas reales a través de conversaciones activas con joyerías locales, quienes han manifestado disposición para prestar sus joyas durante el desarrollo. Esta alianza tiene un impacto directo y significativo en varias etapas del proyecto.

10.1 Digitalización de Joyas Reales para el Dataset

La creación de modelos 3D de alta fidelidad es uno de los cuellos de botella más comunes en proyectos de AR para joyería. Contar con las piezas físicas permite usar técnicas de fotogrametría (captura de múltiples fotografías desde distintos ángulos para reconstruir el modelo 3D) o simplemente servir como referencia visual durante el modelado manual en herramientas CAD. Esto garantiza que los gemelos digitales resultantes sean geométricamente precisos y reflejen fielmente el brillo, las proporciones y los detalles de engaste de las joyas reales de cada joyería aliada.

10.2 Validación Realista del Sistema AR

Disponer de las piezas físicas durante las pruebas permite una validación cualitativa directa: el equipo puede comparar en tiempo real la joya real sobre el dedo o la oreja con la joya virtual superpuesta mediante la aplicación. Este tipo de prueba lado a lado es el estándar de evaluación usado por plataformas comerciales como mirrAR y es muy difícil de replicar sin acceso a los objetos reales. Los resultados de estas pruebas permiten ajustar con precisión los parámetros PBR (metalicidad, rugosidad, color base) hasta lograr una correspondencia visual convincente.

10.3 Diversidad del Catálogo y Casos de Uso

El acceso a colecciones de joyerías locales permite trabajar con una variedad de piezas representativas del mercado real: anillos de compromiso, aretes colgantes, aretes de argolla, anillos de cóctel, entre otros. Esta diversidad es fundamental para que el sistema sea robusto, ya que cada tipo de pieza plantea retos distintos de tracking y renderizado. Un anillo delgado tipo solitario exige mayor precisión de escala que un anillo grueso de cóctel; un arete colgante largo requiere manejo de física de movimiento que un arete de botón no necesita.

10.4 Impacto en la Relevancia del Proyecto

Más allá del beneficio técnico, estas alianzas le dan al proyecto una dimensión de impacto local y aplicabilidad real que va más allá del ejercicio académico. Las joyerías locales, generalmente pequeñas y medianas empresas, no tienen acceso a las plataformas comerciales de AR por sus altos costos. Una solución de código abierto desarrollada con sus propias piezas podría eventualmente ser adoptada por ellas para mejorar su presencia digital, cerrando el círculo entre la investigación y el beneficio para el ecosistema comercial local.

Referencias

1. Dataintel. (2024). Jewelry Try-On AR App Market Research Report 2033. <https://dataintel.com/report/jewelry-try-on-ar-app-market>
2. Fashionbi. (2025). Beyond the Mirror: How AR Is Transforming Fashion, Beauty, Jewellery, and Watches in 2025. <https://www.fashionbi.com/insights/beyond-the-mirror-how-ar-is-transforming-fashion-beauty-jewellery-and-watches-in-2025>
3. mirrAR. (2025). Virtual Try-On: Transforming Jewelry Shopping with AR Technology. <https://www.mirrAR.com/blogs/virtual-try-on-transforming-jewelry-shopping-with-ar-technology>
4. Banuba. (2025). Build Augmented Reality Apps with Flutter – 2025 Guide. <https://www.banuba.com/blog/flutter-ar-guide>
5. Aubergine Solutions. (2025). Creating Flutter Augmented Reality (AR) App. <https://www.aubergine.co/insights/integrating-augmented-reality-into-flutter-a-complete-guide>
6. Google. (2024). MediaPipe Hands documentation. <https://ai.google.dev/edge/mediapipe/solutions/guide>
7. Lars, C. (2022). ar_flutter_plugin – GitHub. https://github.com/CariusLars/ar_flutter_plugin
8. hlefe. (2024). ar_flutter_plugin_2. https://pub.dev/packages/ar_flutter_plugin_2
9. Khronos Group. (2024). PBR – Physically Based Rendering in glTF. <https://www.khronos.org/glTF/pbr>
10. Snap Inc. (2024). Gemstone Rendering – Snap for Developers. <https://developers.snap.com/lens-studio/features/graphics/materials/gem-rendering>
11. Yasser, A. (2024). Understanding Impeller – Flutter's Modern Rendering Engine. Medium. <https://medium.com/@ImAmmarYasser/understanding-impeller-flutters-modern-rendering-engine-13e04ed5fde4>